Predictive Deadlock Warning Using Machine Learning

An early-warning study for unsafe resource-allocation states, using a simulated operating system, Banker's algorithm as ground truth, and two supervised classifiers.

Course Operating Systems, Department of Information Technology, CBIT
Team Revanth Reddy Lingala (160124737-115), Padigela Srujan (160124737-125)
Report date 14 October 2026

Status: all results below are measured on the 150 validation runs. Model settings and alarm thresholds were also chosen on those runs, so the numbers are slightly optimistic. The 150 test runs are sealed and will be scored once, after the method is final.

ABSTRACT

Banker's algorithm can tell an operating system whether its current resource allocation is safe, but not whether it is about to become unsafe. We ask whether supervised machine learning can give that warning in advance. We built a resource-allocation simulator with six workload scenarios, labelled every step with Banker's safety check and a separate deadlock detector, and generated 1,002 runs of 2,000 steps. From each step we computed 43 features that use only the present and the past, and trained Logistic Regression and Random Forest to predict whether the state will turn unsafe within the next 20 (or 5) steps. On held-out validation runs, Random Forest reaches a PR-AUC of 0.553 at the 20-step horizon against 0.454 for Logistic Regression and a no-skill level of 0.070; the gap of 0.099 has a 95% interval of 0.067 to 0.131. Set to warn before 80% of unsafe onsets, Random Forest keeps the alarm on for 9.1% of the time against 12.5% for Logistic Regression. Warning five steps ahead is much harder (PR-AUC 0.289). Removing the features derived from Banker's algorithm lowers performance only slightly, so the models learn more than a repackaged safety check.

1. Introduction

A deadlock happens when a group of processes each wait for resources that another member of the group holds, so none of them can continue. Operating systems handle this in three classic ways: prevent it by design, avoid it by checking each request with Banker's algorithm, or detect it after it happens and recover. All three describe the system as it is now.

Banker's algorithm separates *safe* states, where some order exists in which every process can finish, from *unsafe* states, where no such order is guaranteed. An unsafe state is not yet a deadlock, but it is the point at which the system loses its guarantee. A warning that arrives before that point would give a scheduler time to act, for example by delaying new admissions or large requests.

Research question. Can supervised machine learning predict an upcoming unsafe resource-allocation state from current system-state features and recent request patterns, and how reliably and how early can it warn?

The contribution is a controlled comparative study, not a new algorithm: a tested simulator, a leakage-controlled dataset split by whole runs, and an evaluation of Logistic Regression and Random Forest that measures not only how well each ranks risk but how early it warns and how often it raises an alarm.

2. Background and related work

Avoidance and detection. Dijkstra introduced the Banker's algorithm [1]; Habermann extended it to several resource types [2]. Coffman, Elphick and Shoshani [4] and Holt [5] set out the detection model in which a waiting process can proceed once its pending request fits in the free resources. Gold showed that safety checking is easy in our class of systems and hard in others [6], which is why exact labels for millions of states are cheap to compute here. Holt also noted that "unsafe" is a conservative signal rather than proof of harm [3].

Safety as classification. Work by Reveliotis and colleagues treats deadlock avoidance as drawing a boundary between safe and unsafe states and shows that a linear boundary is not sufficient in general [11, 12]. That motivates comparing a linear model with a non-linear one.

Learned deadlock prediction. The closest prior work forecasts thread behaviour in multithreaded programs with neural networks and Markov chains and then checks the forecast for deadlock [8], or tracks developing

deadlocks with a recurrent network and chooses a resolution with a second network, reporting about 80% accuracy and over 82% of deadlocks prevented [9]. A cloud study uses linear regression to forecast virtual-machine "deadlock", which there means overload [10]. These studies report accuracy, use small setups and do not measure warning lead time or false alarms.

Evaluation. With rare positives, precision-recall curves are more informative than ROC curves [13]; event-style predictions need event-level metrics rather than per-step scores [14]; and leakage between training and test data has invalidated many published ML results [15]. Our design follows all three.

Gap. We found no study that evaluates early warning of Banker's-unsafe onsets at the event level, with run-level splits and confidence intervals. This report fills that narrow gap.

3. Method

```
simulate  →  label with Banker's  →  43 features  →  split by run  →  train LR / RF  →  set threshold  →  evaluate
fake OS      + deadlock detector     (past only)       70 / 15 / 15     (train runs)       (validation)       (validation)
```

3.1 Simulator

The simulator models n process slots (20 to 60) competing for m resource types (3 to 6). It keeps the textbook structures: `Available`, `Allocation`, the declared maximum `Max`, `Need = Max − Allocation`, and the pending `Request` of each waiting process. One step is one event: a randomly chosen runnable process requests resources, works a turn, finishes, or leaves early. A request that fits in the free resources is granted; otherwise the process waits, and waiting requests are retried in the order they blocked whenever resources are released. Banker's algorithm never decides a grant. It only labels the state after each step, so runs can drift into unsafe states and the dataset contains the transitions we want to predict. Each run lasts 2,000 steps.

A flaw we corrected. If every process eventually requests its full declared maximum and releases resources only when it finishes, an unsafe state can never become safe again and always ends in deadlock. Each run would then contain at most one event, and "unsafe" and "deadlock" would be the same label shifted in time. We therefore gave each process a hidden true demand at or below its declared maximum, let processes leave early with a small probability, and resolve a deadlock by aborting the deadlocked process holding the most units, one per step, so a run continues and can contain many episodes. A test that switches these mechanisms off confirms that, without them, no run ever returns from unsafe to safe. With them, between 14% and 66% of unsafe onsets end in deadlock, depending on the scenario (Table 1), so the two labels are now genuinely different.

A second, subtler fix: a process still short of its hidden demand sometimes works a turn without requesting. Otherwise a "work" turn would reveal that a process had reached its hidden demand, leaking information the operating system could not know.

Scenario	What changes	Onsets (train)	P(deadlock | onset)
Normal	Moderate claims, wide range of true demand	647	0.175
Sudden spike	Three bursts of larger, faster requests	830	0.135
Resource scarcity	Two resource types cut to 60% capacity	1061	0.451
High contention	Claims concentrated on two resource types	1359	0.429
Gradual build-up	Near-full true demand, small requests, long holds	447	0.657
Mixed random	Each run mixes the settings above	870	0.625

Table 1. The six workload scenarios. Each is the same simulator with different settings, fixed on development seeds and checked on separate pilot seeds before the dataset was generated. Onset counts and the share of onsets that end in deadlock are from the training split.

3.2 Labels

After every step, Banker's safety algorithm labels the state safe or unsafe, and a separate detector, which works from pending requests, labels it deadlocked or not. Tests confirm that every deadlocked state is also unsafe, and that unsafe states without deadlock exist. An *onset* is a step where the state turns from safe to unsafe; two unsafe episodes separated by fewer than three safe steps count as one. The prediction target at step t is 1 if an onset occurs in steps t+1 to t+H, for H = 20 (main) and H = 5.

A step is a *prediction point* only if the state is safe now (a warning is only useful before the event), at least 20 steps of history exist, and the run continues for at least H more steps, so no unknown future is labelled "no onset".

3.3 Features

Each step is described by 43 numbers in seven groups: system size; resource availability; process status (running, waiting, holding); declared remaining demand; recent request history over the last 5 and 20 steps (requests, grants, waits, releases, completions, arrivals, and their trends); recent safety history (how recently the state was unsafe); and four quantities taken from the Banker's reduction itself, such as the share of processes that could finish immediately. Most are ratios, so they mean the same at any system size.

Leakage was prevented by construction. The feature code receives only what an operating system could observe: no scenario name, seed, simulator setting or hidden demand. Tests recompute features at step t with every later step deleted, and with every later step replaced by random noise; the features do not change.

3.4 Dataset and splits

We generated 1,002 runs, an equal number per scenario, and split them by whole run (Table 2). Neighbouring steps of one run are nearly identical, so splitting by row would test the models on near-copies of their training data. Each split uses its own seed branch, and no seed used during development appears in the dataset.

	Train	Validation	Test
Runs (per scenario)	702 (117)	150 (25)	150 (25)
Rows (steps)	1,404,000	300,000	300,000
Prediction points, H = 20	1,185,144	256,449	244,746
Positive share, H = 20	7.7%	7.0%	9.4%
Positive share, H = 5	2.2%	2.0%	2.6%
Unsafe onsets	5,214	1,018	1,318
Used for	Fitting the models	Settings, thresholds, all results here	Sealed until the method is final

Table 2. Dataset by split. Positive share is the fraction of prediction points with an onset coming within the horizon.

3.5 Models and training

Logistic Regression is the linear baseline: features are standardised, and the penalty strength C is chosen from {0.01, 0.1, 1, 10}. **Random Forest** uses 200 trees, with the minimum leaf size chosen from {5, 20, 50}. Both use balanced class weights, because positives are rare and an unweighted model learns mostly to say "no". Training uses every fifth prediction point (about 237,000 rows per horizon), since adjacent steps add little new information. For each model the setting with the best validation PR-AUC is kept. The model output is a risk score used for ranking; because of the class weighting it is not a calibrated probability.

To turn scores into alarms, each model gets a threshold: the highest score cut-off that still raises an alarm before at least 80% of validation onsets. Fixing the catch rate makes the models comparable on how much alarming they need to achieve it.

3.6 Evaluation

Ranking quality is measured by PR-AUC, always shown with its no-skill value (the positive share), with ROC-AUC as a secondary measure. **Warning quality** is measured per onset: an onset is *warned* if an alarm occurs in the H steps before it (an alarm at the onset itself is too late); *lead time* is the distance from the first such alarm to the onset; *alarm share* is the fraction of prediction points with the alarm on, which measures the cost of the warnings. **Uncertainty** is measured by a bootstrap that resamples whole validation runs 1,000 times; model differences use the same resampled runs for both models. **Ablation** retrains the models with feature groups removed.

4. Results

4.1 How well the models rank risk

Horizon	Model	No-skill	PR-AUC (95% interval)	ROC-AUC
20 steps	Logistic Regression	0.070	0.454 (0.403–0.503)	0.917
20 steps	Random Forest	0.070	0.553 (0.508–0.595)	0.935
20 steps	*Difference, RF − LR*		**+0.099 (0.067–0.131)**	
5 steps	Logistic Regression	0.020	0.185 (0.162–0.213)	0.919
5 steps	Random Forest	0.020	0.289 (0.254–0.325)	0.934
5 steps	*Difference, RF − LR*		**+0.105 (0.068–0.138)**	

Table 3. Ranking quality on validation runs, with 95% intervals from 1,000 run-level bootstrap resamples.

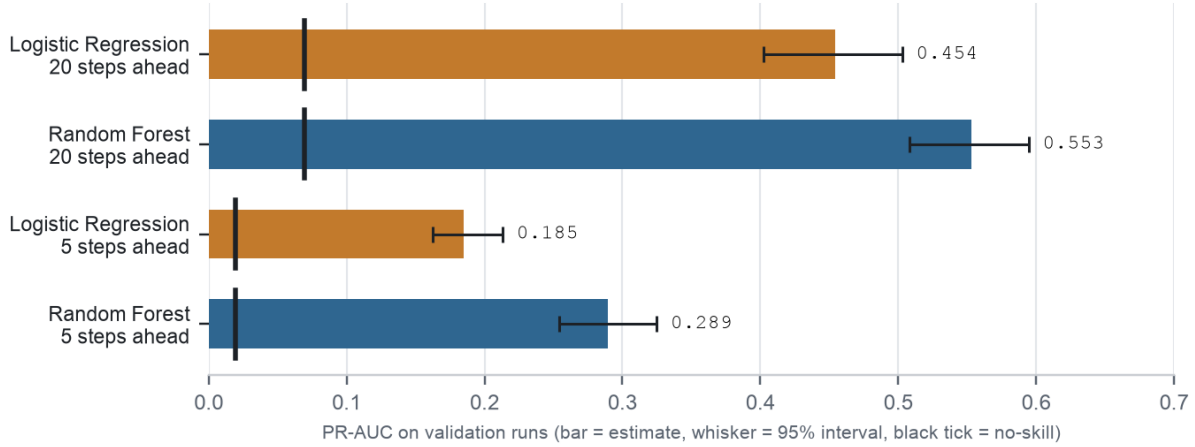


Figure 1. PR-AUC with 95% intervals. Both models sit far above the no-skill level at both horizons.

Both models rank risk far better than chance: at the 20-step horizon Random Forest scores 0.553 against a no-skill level of 0.070, about eight times higher. Random Forest beats Logistic Regression by 0.099 (95% interval 0.067 to 0.131), and the interval excludes zero at both horizons, so the difference is not an accident of which runs were sampled. ROC-AUC is above 0.9 for both models; it looks high because the many easy negative steps make false alarms a small fraction of all negatives, which is why PR-AUC is the main measure.

4.2 How well the models warn

Horizon	Model	Onsets warned	Warned ≥ 5 / 2 steps early	Alarm share	Alarm steps correct	First warning at full horizon
20 steps	Logistic Regression	80%	76%	12.5%	37%	59%
20 steps	Random Forest	80%	75%	9.1%	48%	47%
20 steps	*Always alarm (reference)*	100%	98%	100.0%	7%	79%
5 steps	Logistic Regression	80%	78%	12.1%	12%	87%
5 steps	Random Forest	80%	78%	8.8%	16%	83%
5 steps	*Always alarm (reference)*	100%	100%	100.0%	2%	97%

Table 4. Warning quality at the main threshold, which is set on validation to warn before 80% of onsets. "Warned early" means at least 5 steps ahead for H = 20 and at least 2 for H = 5. The always-alarm row is a reference, not a model.

At the same 80% catch rate, Random Forest keeps the alarm on for 9.1% of the time against 12.5% for Logistic Regression, a reduction of 3.4 percentage points (95% interval 2.2 to 4.8). When its alarm is on, an onset really is coming 48% of the time, against a 7% base rate. 75% of onsets are warned at least five steps ahead, and for about half of warned onsets the alarm was already on at the earliest point that counts, so the true lead is often longer than the 20-step window can show. At the 5-step horizon the picture is weaker: the alarm is on 8.8% of the time and only 16% of alarm steps are correct.

A pitfall in event metrics. Two common event metrics, alarm precision and false alarms per 1,000 steps, count continuous bursts of alarm as single alarms. A rule that alarms on every step is then a single "correct" alarm per safe stretch and scores better on both than either model, although it is useless. We therefore report alarm share, which exposes it (100%), and show the always-alarm row as a reference.

4.3 Results by scenario

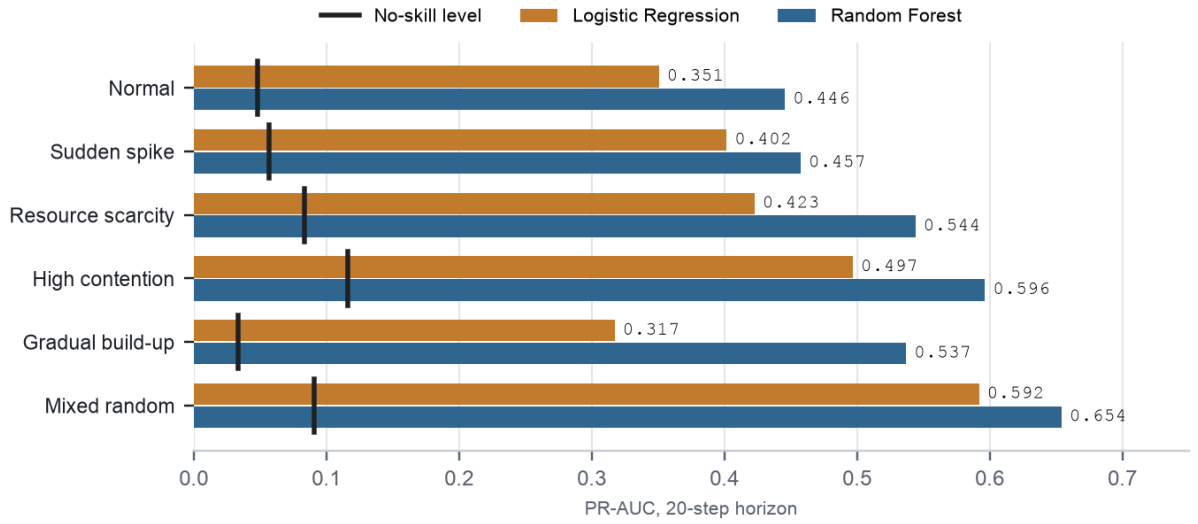


Figure 2. PR-AUC by scenario at the 20-step horizon.

Random Forest is ahead in every scenario. The share of onsets warned ranges from 69% (Normal) to 91% (Mixed random). Normal workloads have few, subtle onsets, and sudden spikes give little build-up to detect. The gradual build-up scenario shows the largest gain from Random Forest, consistent with slowly accumulating pressure being visible in combinations of features rather than in any single one.

4.4 What the models depend on

Horizon	Model	All 43 features	Without Banker-derived (39)	Without safety history (40)	Without both (36)
20 steps	Logistic Regression	0.454	-0.016 (-0.029 to -0.002)	+0.001 (-0.007 to +0.009)	-0.019 (-0.035 to -0.002)
20 steps	Random Forest	0.553	-0.035 (-0.046 to -0.025)	+0.002 (-0.003 to +0.007)	-0.037 (-0.049 to -0.026)
5 steps	Logistic Regression	0.185	-0.023 (-0.045 to -0.007)	+0.004 (-0.001 to +0.008)	-0.024 (-0.049 to -0.006)
5 steps	Random Forest	0.289	-0.032 (-0.040 to -0.024)	+0.000 (-0.006 to +0.007)	-0.034 (-0.045 to -0.023)

Table 5. Ablation: change in validation PR-AUC when feature groups are removed and the models retrained, with 95% bootstrap intervals for the change.

Removing the four Banker-derived features lowers PR-AUC by 0.016 to 0.035, a small but real drop. Random Forest still scores about 0.52 at the 20-step horizon without them, so most of what it learns comes from ordinary state and history features rather than from repackaging Banker's calculation. Removing recent safety history changes nothing measurable, so the models are not relying on the shortcut "the state was unsafe a moment ago".

5. Discussion

- **Machine learning gives real but imperfect early warning.** At an 80% catch rate, roughly half of alarm steps are correct, against a base rate of 7%. One onset in five is still missed.
- **The non-linear model is better.** Random Forest beats Logistic Regression in ranking and needs about a quarter less alarm time for the same catch rate, consistent with the theoretical result that a linear boundary is not enough to separate safe from unsafe states.
- **Lead time matters as much as accuracy.** The 20-step task is much easier than the 5-step task. Pressure that builds up is visible well in advance; the final few steps before an onset are dominated by individual random requests.

- **Evaluation choices change conclusions.** ROC-AUC alone would make both models look near-perfect, and episode-based alarm metrics would rank a useless always-on alarm first. PR-AUC and alarm share give the honest picture.

6. Limitations

- **Synthetic data.** All results hold for our simulator only. Real operating systems rarely know each process's declared maximum, which Banker's algorithm requires.
- **Validation results.** Settings and thresholds were chosen on the same runs used for evaluation, and the intervals do not include that selection effect. Test results may be slightly lower.
- **No simple reference rule yet.** The specification excludes rule-based predictors, so we have not measured how much the models improve on, for example, a threshold on a single feature. We have asked for guidance on reporting such a reference.
- **Lead time is capped** at the horizon by construction, and the merging of brief safe gaps looks one or two steps ahead, affecting a few hundred of 1.4 million rows.
- **Limited scale.** Systems have 20 to 60 processes; behaviour on larger systems is untested.

7. Conclusion and future work

Within a controlled simulator, supervised learning can warn of most unsafe onsets well before they happen. Random Forest is clearly better than Logistic Regression: at the 20-step horizon it warns before 80% of onsets with the alarm on 9.1% of the time, and the advantage holds across all six workloads and under resampling. Next steps are to compare against a simple reference rule and estimate the best achievable score by simulating many futures from the same state, test on larger systems and on held-out scenarios, calibrate the risk scores, and then score the sealed test runs once and reproduce the full pipeline on AWS EC2 as the specification requires.

Implementation and reproducibility

The code is in Python (NumPy, pandas, scikit-learn). The simulator, Banker's algorithm, deadlock detector, features and labels are in `src/pdw/`; dataset generation is `scripts/generate_dataset.py`; the modelling steps are separate files in `model/` (loading, feature selection, training, prediction, evaluation, event metrics, bootstrap, ablation). 133 automated tests cover the textbook Banker's and detection examples, agreement between two independent Banker's implementations on 10,000 random states, the invariant that deadlocked implies unsafe, reproducibility of every run from its seed, the absence of future information in the features, and the event metrics. Every run is seeded, so the dataset and results can be regenerated exactly. All design decisions and their reasons are listed in `DECISIONS.md`.

References

Bibliographic details were checked against publisher and index records during the project audit; they will be confirmed against the originals before final submission.

[1] E. W. Dijkstra, "Cooperating Sequential Processes" (EWD123), 1965; in F. Genuys (ed.), *Programming Languages*, Academic Press, 1968, pp. 43–112.

[2] A. N. Habermann, "Prevention of system deadlocks," *Communications of the ACM*, 12(7), 373–377, 385, 1969. doi:10.1145/363156.363160

[3] R. C. Holt, "Comments on prevention of system deadlocks," *Communications of the ACM*, 14(1), 36–38, 1971. doi:10.1145/362452.362486

[4] E. G. Coffman, M. Elphick, A. Shoshani, "System deadlocks," *ACM Computing Surveys*, 3(2), 67–78, 1971. doi:10.1145/356586.356588

[5] R. C. Holt, "Some deadlock properties of computer systems," *ACM Computing Surveys*, 4(3), 179–196, 1972. doi:10.1145/356603.356607

[6] E. M. Gold, "Deadlock prediction: easy and difficult cases," *SIAM Journal on Computing*, 7(3), 320–336, 1978. doi:10.1137/0207027

[7] A. Silberschatz, P. B. Galvin, G. Gagne, *Operating System Concepts*, Wiley (edition to be stated).

[8] S. M. Babamir, E. Hassanzade, M. Azimpour, "Predicting potential deadlocks in multithreaded programs," *Concurrency and Computation: Practice and Experience*, 27(17), 5261–5287, 2015. doi:10.1002/cpe.3568

[9] Ghorbani and S. M. Babamir, "Runtime deadlock tracking and prevention of concurrent multithreaded programs: a learning-based approach," *Concurrency and Computation: Practice and Experience*, 32(10), e5324, 2020. doi:10.1002/cpe.5324

[10] H. Le Ngoc, H. Tran Cong, "Enhancing load balancing in cloud computing through deadlock prediction," INISCOM 2023, LNICST, Springer, pp. 257–274. doi:10.1007/978-3-031-47359-3_19

[11] M. Lawley, S. Reveliotis, "Deadlock avoidance for sequential resource allocation systems: hard and easy cases," *International Journal of Flexible Manufacturing Systems*, 13, 385–404, 2001. doi:10.1023/A:1012203214611

[12] M. Ibrahim, S. Reveliotis, A. Nazeem, "Maximal linear deadlock avoidance policies for sequential resource allocation systems: characterization, computation and approximation" (venue to be confirmed).

[13] T. Saito, M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLOS ONE*, 10(3), e0118432, 2015. doi:10.1371/journal.pone.0118432

[14] N. Tatbul et al., "Precision and recall for time series," *Advances in Neural Information Processing Systems* (NeurIPS), 2018.

[15] S. Kapoor, A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, 4(9), 100804, 2023. doi:10.1016/j.patter.2023.100804